



University
of Glasgow

W4-06: Batch vs. Streaming

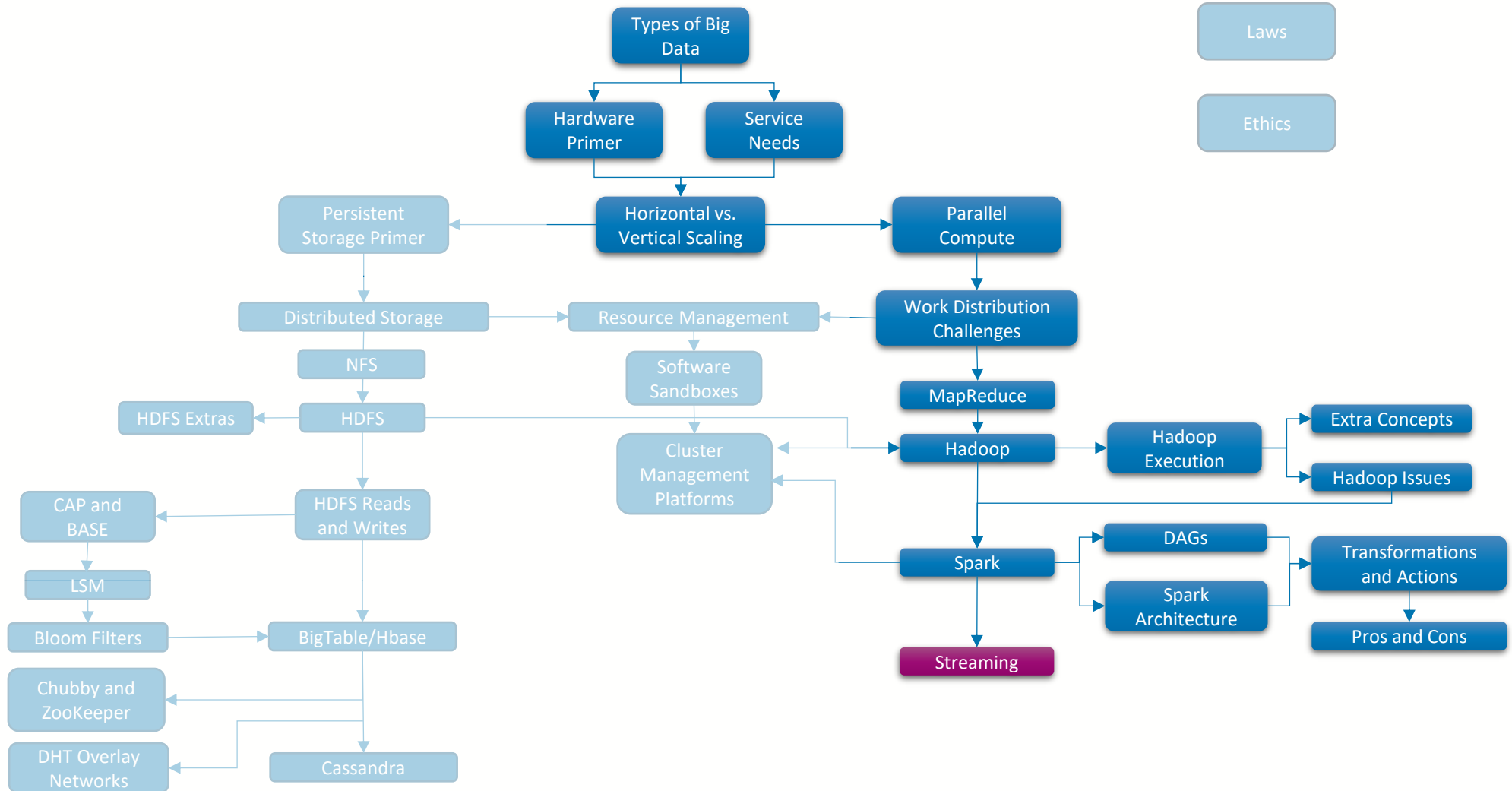
COMPSCI4064 (H) and COMPSCI5088 (M)

Dr. Richard McCreddie

**WORLD
CHANGING
GLASGOW**

**A WORLD
TOP 100
UNIVERSITY**

Where Are We?





Batch vs. Streaming

- An important big data dimension to consider is whether we are working with largely static data, or new data is arriving over time
- The underlying Big Data solution you use will be different in these two cases
 - **Static Data:** Tackled with Batch processing, where large blocks of that data are analysed at once, usually with multiple blocks being processed concurrently, that are subsequently merged together
 - Cares about **completion time**
 - **Stream Data:** Tackled with Stream processing, where items are processed individually (or in very small blocks) within a pipeline of stages, where multiple items can be processed concurrently within the pipeline
 - Cares about **processing latency** per item



What makes Streaming Challenging?

- The core problems that streaming introduces are two-fold:
 1. We do not have discrete blocks of data of roughly equal size to process
 - Load-balancing is more difficult
 - Velocity/Input rate is variable and usually tied to real-world events
 2. Usually item end-to-end processing time is critical because there is a human user waiting for the result
 - We need to be more concerned that we achieve consistent processing latencies
 - Buffering during task-to-task transfers can be problematic
 - Bottlenecks/back pressure can cause failures

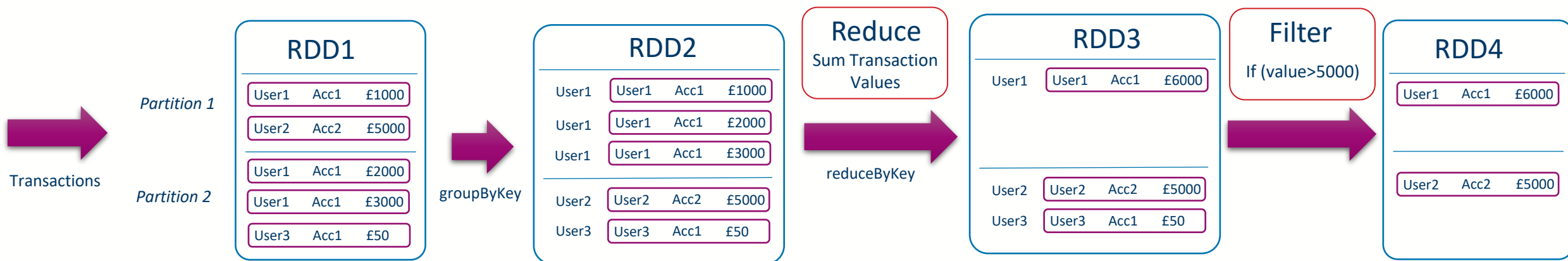


Batch Vs. Streaming Example

- Imagine you are working at a bank and are responsible for identifying suspicious transactions made by your customers
 - If a retail customer transfers more than £5,000 in a single day than those transactions should be checked
- How to we do this?
 - In a **Batch** form?
 - In a **Streaming** form?

A Batch Solution (Spark)

- We break our financial transactions into batches/partitions
- Group our transactions based on who made them (groupByKey)
- Sum the transactions for a single day per user (reduce)
- Filter out users that have spent less than £5000

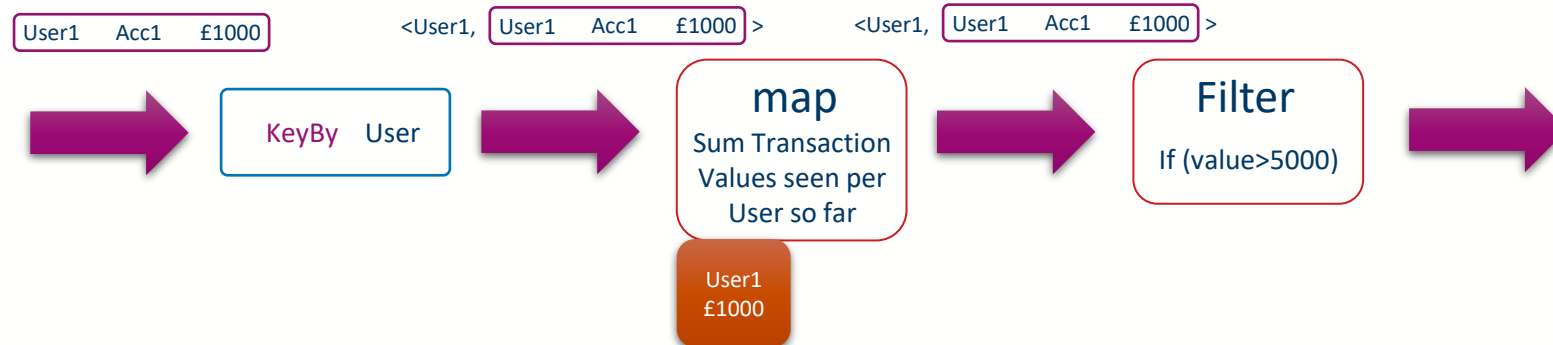


The issue with this solution is that we need to wait until the end of the day to run the check



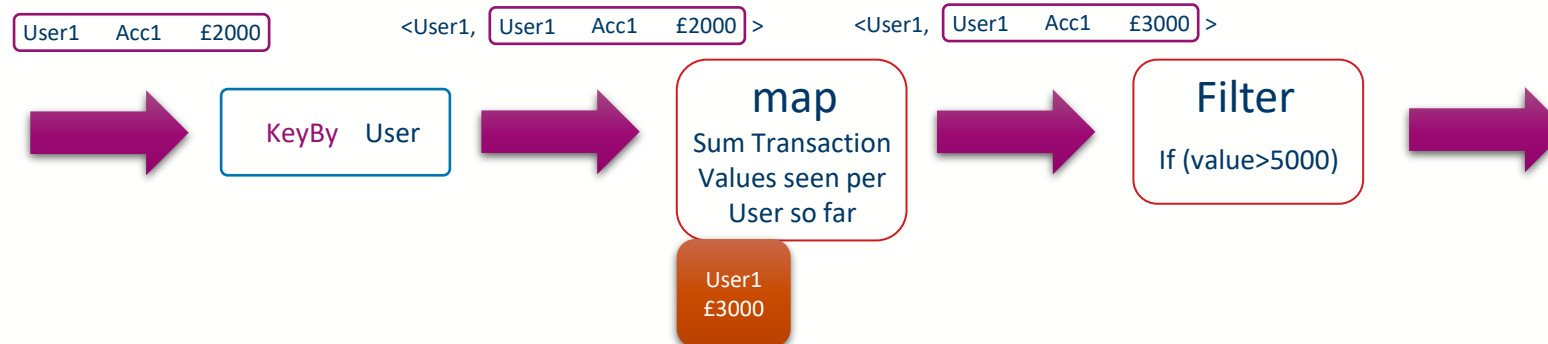
A Streaming Solution

- Assign the User as the key for each transaction
- Perform a map transformation, that sums the total transaction value for each user seen so far by the map
 - The map maintains a counter to store transaction value for each user
 - The counter gets reset at the end of each day
- Filter out users that have spent less than £5000



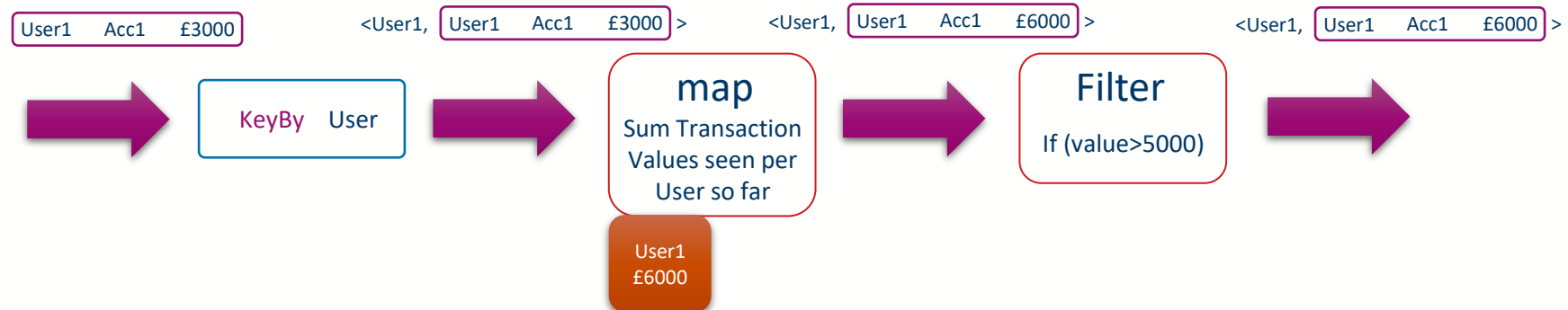
A Streaming Solution

- Assign the User as the key for each transaction
- Perform a map transformation, that sums the total transaction value for each user seen so far by the map
 - The map maintains a counter to store transaction value for each user
 - The counter gets reset at the end of each day
- Filter out users that have spent less than £5000



A Streaming Solution

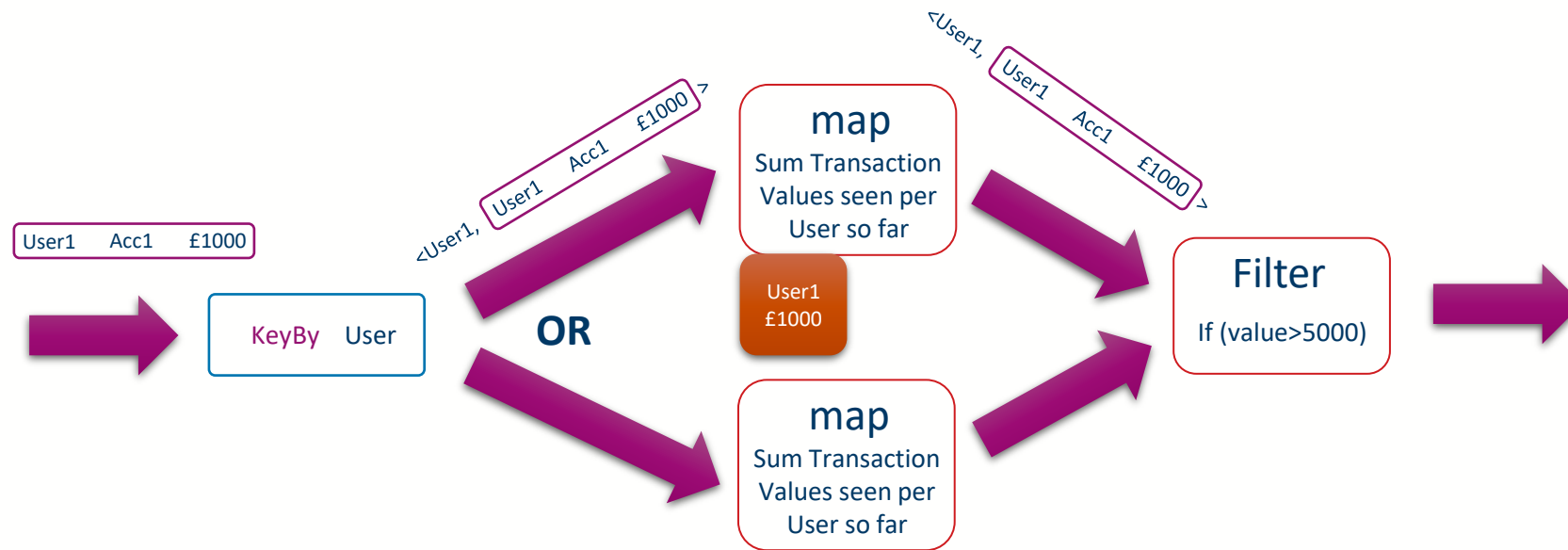
- Assign the User as the key for each transaction
- Perform a map transformation, that sums the total transaction value for each user seen so far by the map
 - The map maintains a counter to store transaction value for each user
 - The counter gets reset at the end of each day
- Filter out users that have spent less than £5000





Problems with Parallelism

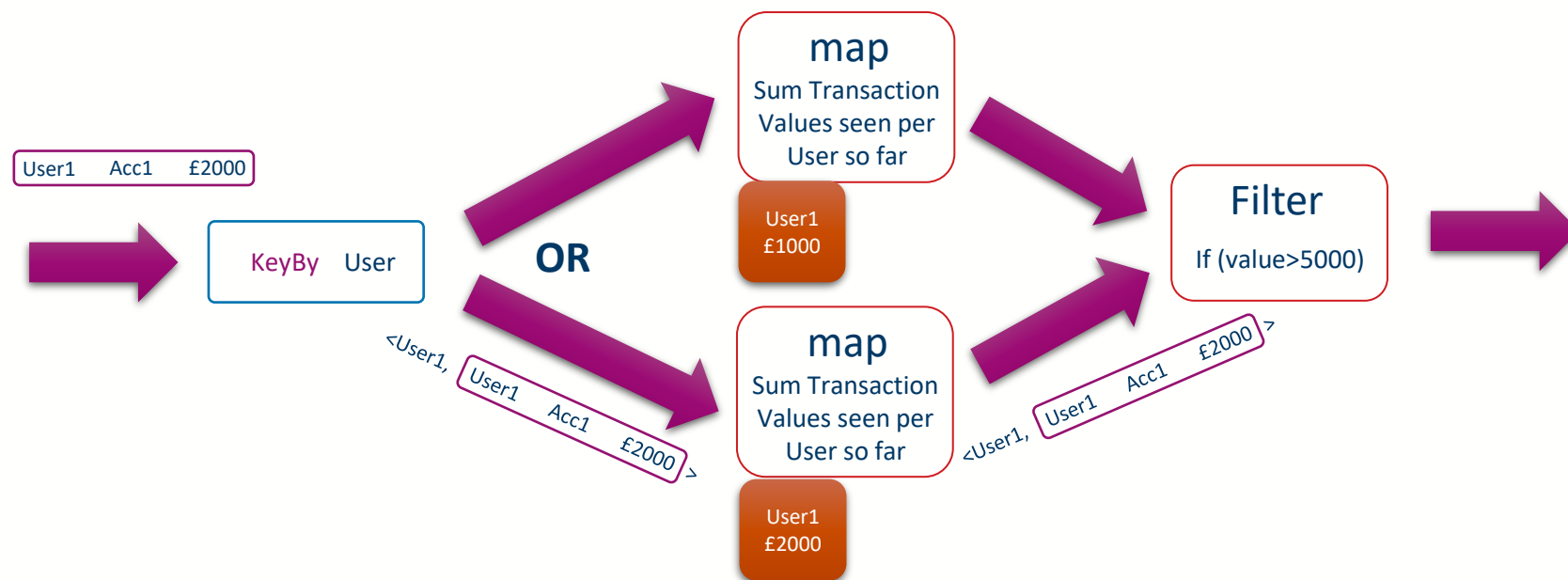
- Lets imagine that we distribute the map function





Problems with Parallelism

- Lets imagine that we distribute the map function



- If we **randomly** assigned transactions to maps, we would get inconsistent results



Stateful Applications

- This type of application is known as a **stateful application**
 - It means that one or more of the tasks has some **persistent state that changes over time** based on the input it receives

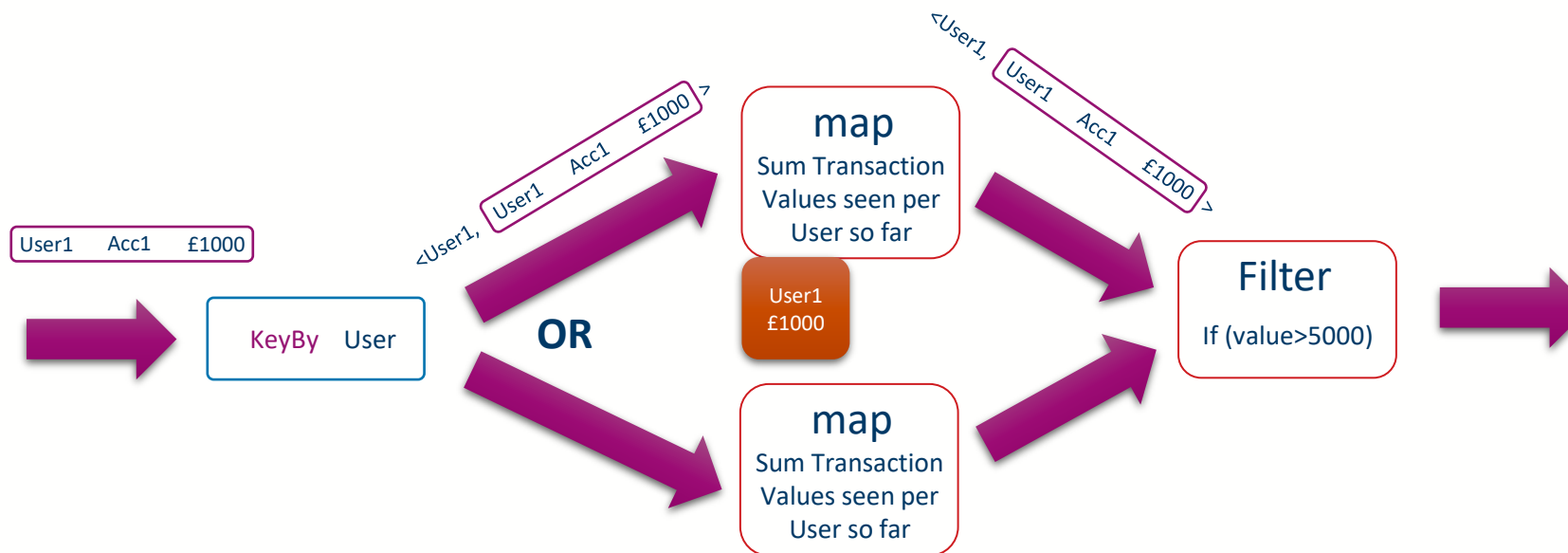


- State causes a lot of problems for streaming systems, as it means that we are forced to route particular transactions to particular task instances to maintain consistent processing



Stateful Applications

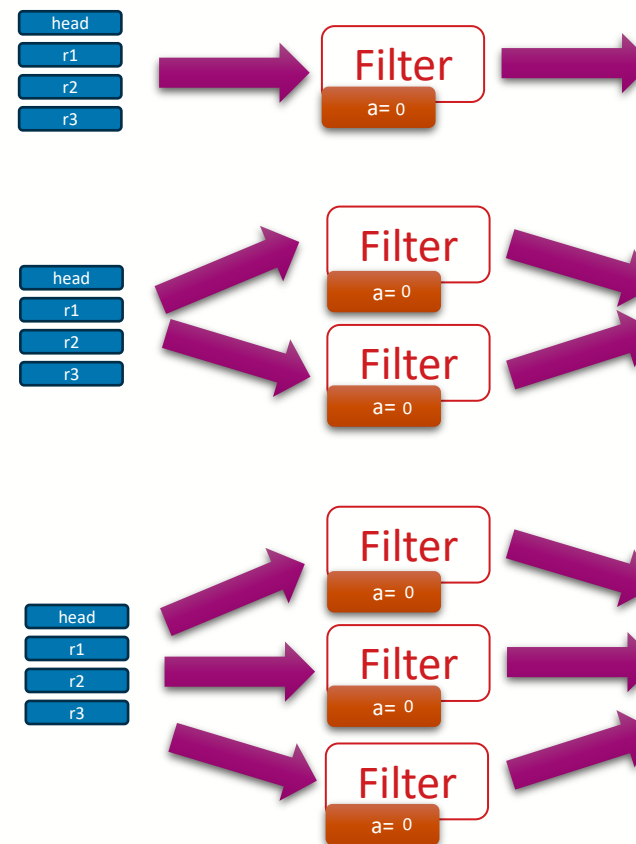
- The typical solution to this problem is to **group your data by some logical key**, such that a single map instance receives all items belonging to that key
 - In this case we can **route** transactions based on the user key



Shared State

- Applications that want to **share state** between instances of a task are even more problematic
 - These are scenarios where there is no key we can use to divide the stream without altering the output

Let's consider a simple example, imagine we are processing a document, where the first record is the header, so we define a filter function that filters out the first record it sees, how does this work at different replication factors?

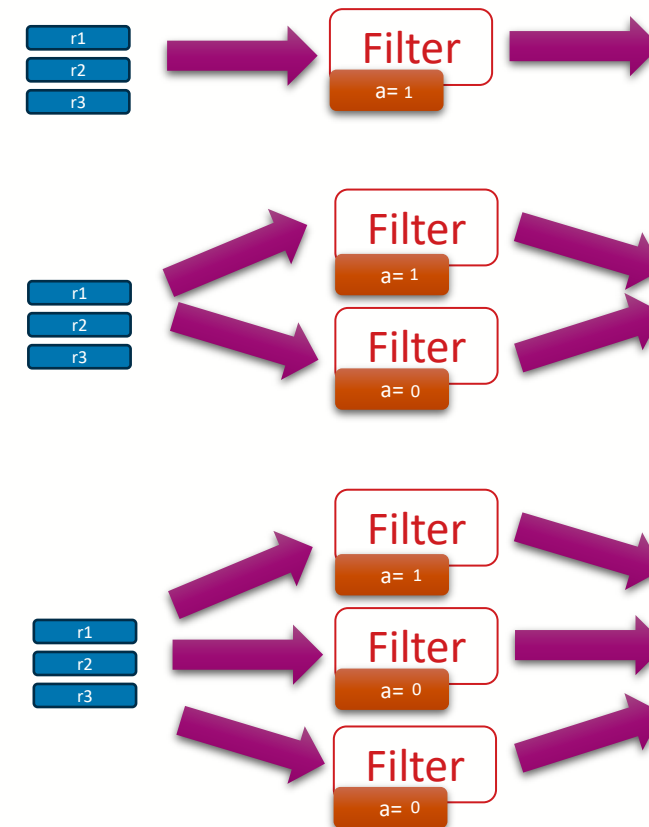




Shared State

- Applications that want to **share state** between instances of a task are even more problematic
 - These are scenarios where there is no key we can use to divide the stream without altering the output

Let's consider a simple example, imagine we are processing a document, where the first record is the header, so we define a filter function that filters out the first record it sees, how does this work at different replication factors?

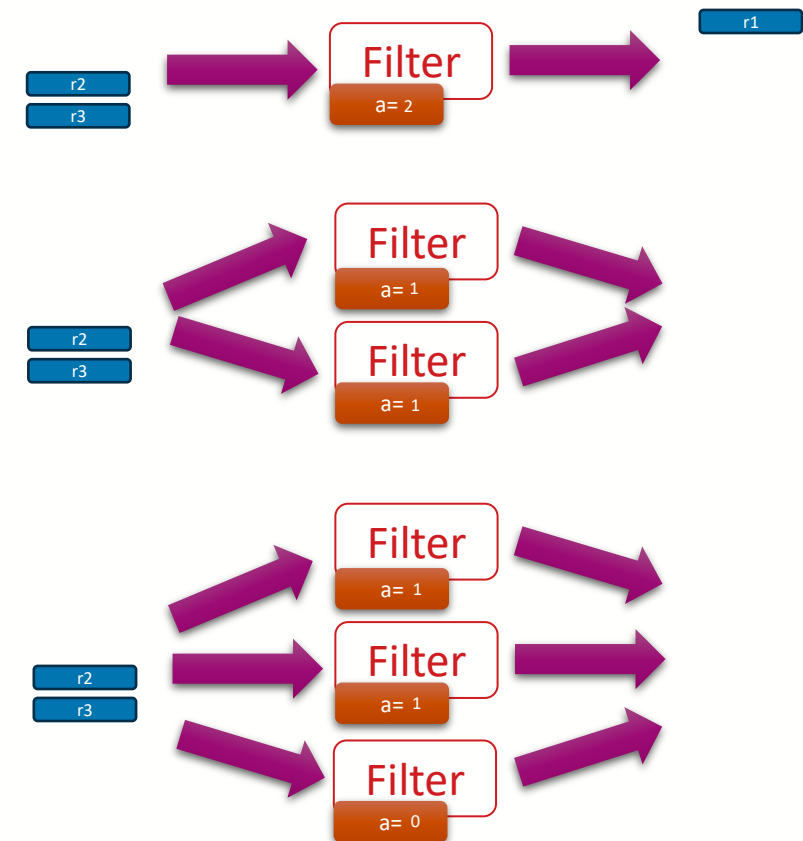




Shared State

- Applications that want to **share state** between instances of a task are even more problematic
 - These are scenarios where there is no key we can use to divide the stream without altering the output

Let's consider a simple example, imagine we are processing a document, where the first record is the header, so we define a filter function that filters out the first record it sees, how does this work at different replication factors?

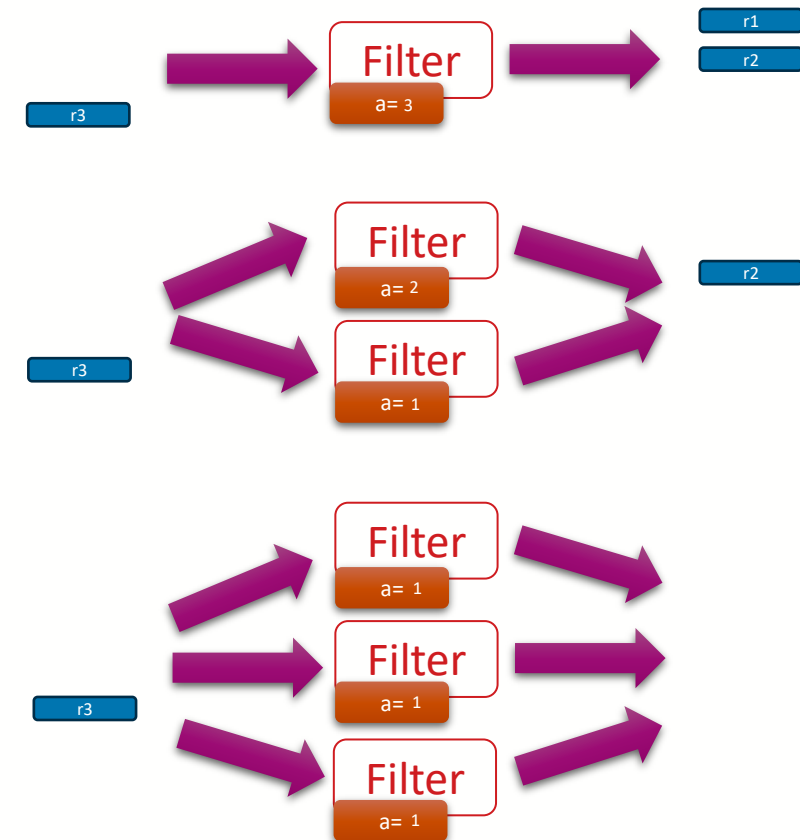




Shared State

- Applications that want to **share state** between instances of a task are even more problematic
 - These are scenarios where there is no key we can use to divide the stream without altering the output

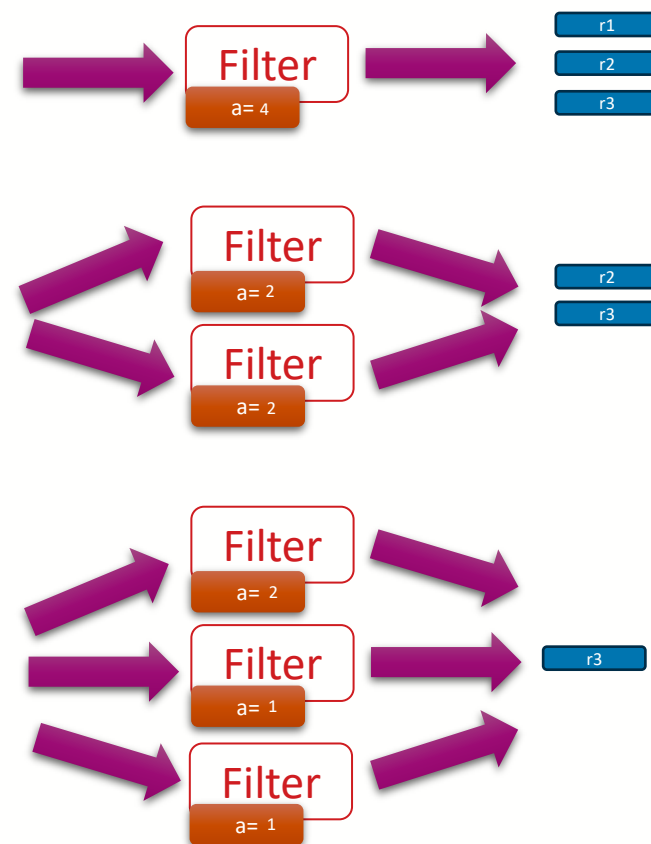
Let's consider a simple example, imagine we are processing a document, where the first record is the header, so we define a filter function that filters out the first record it sees, how does this work at different replication factors?



Shared State

- Applications that want to **share state** between instances of a task are even more problematic
 - These are scenarios where there is no key we can use to divide the stream without altering the output

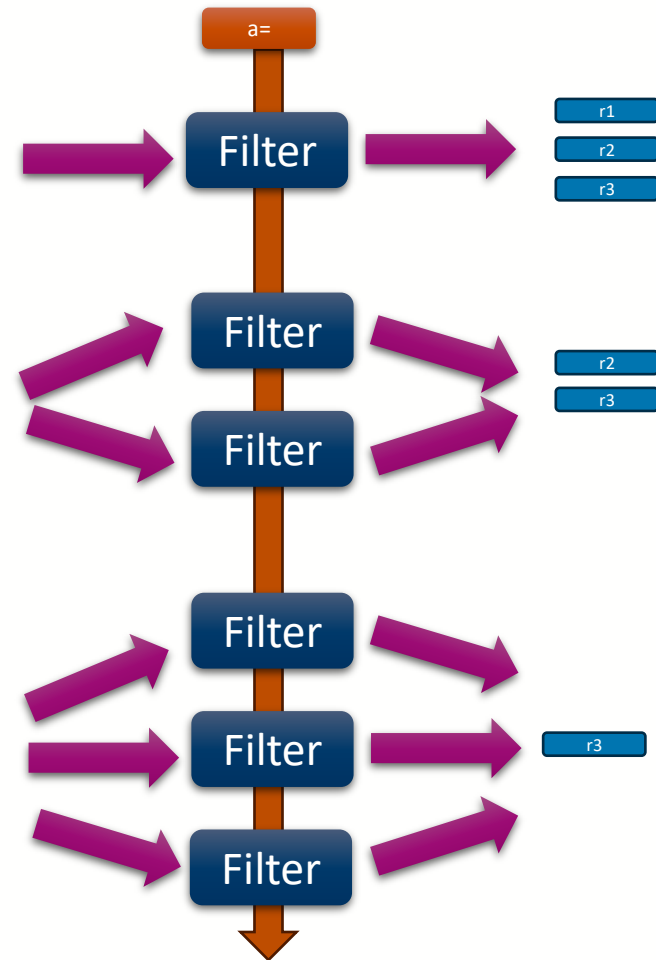
Let's consider a simple example, imagine we are processing a document, where the first record is the header, so we define a filter function that filters out the first record it sees, how does this work at different replication factors?





Shared State

- Applications that want to **share state** between instances of a task are even more problematic
 - These are scenarios where there is no key we can use to divide the stream without altering the output
- If all of the task instances could share their state (a in this case), the application would work consistently
 - However this is **difficult in practice**
 - **Synchronization** of state across the network **takes time** and **processing is blocked** until that completes
 - **Failure** of one task can cause state to become **inconsistent** or can **corrupt** the global state





Other problems with state...

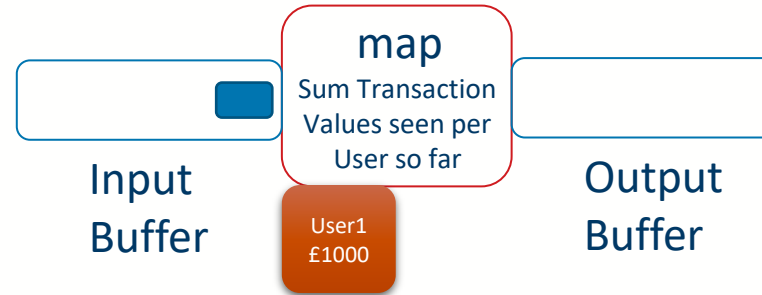
- There are also other challenges that are introduced when we start holding state within tasks
 - **Memory usage** of our tasks **grows** over time as the state grows
 - Can lead to tasks exhausting memory and failing
 - If a task **fails** mid-processing the **state is lost**
 - Difficult to recover from, we would need to re-process all items seen up-to that point to re-generate the state

State management is a major problem for both streaming and batch-based operations, try and avoid including state in transformations where possible!



Buffering

- The other core problem we observe in streaming systems is buffering
 - Imagine each task has an in-buffer and an out-buffer

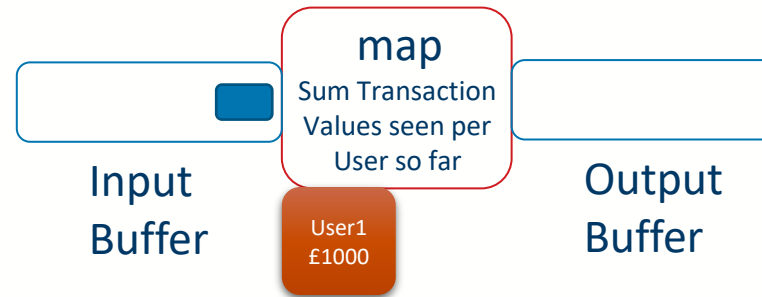


- The task can only process one item at a time, so it takes the first item in the in-tray, and when its done it places the item in the out-tray



Buffering

- The other core problem we observe in streaming systems is buffering
 - Imagine each task has an in-buffer and an out-buffer

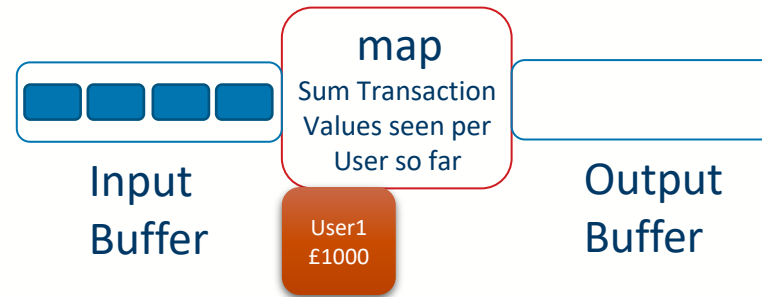


- New items will arrive over time and will be placed in the in-buffer
- If items arrive faster than the task can process them, the buffer will fill



Buffering

- The other core problem we observe in streaming systems is buffering
 - Imagine each task has an in-buffer and an out-buffer

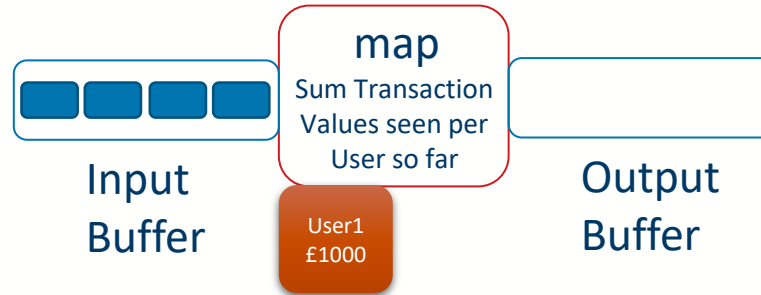


- New items will arrive over time and will be placed in the in-buffer
- If items arrive faster than the task can process them, the buffer will fill



Buffering

- The other core problem we observe in streaming systems is buffering
 - Imagine each task has an in-buffer and an out-buffer

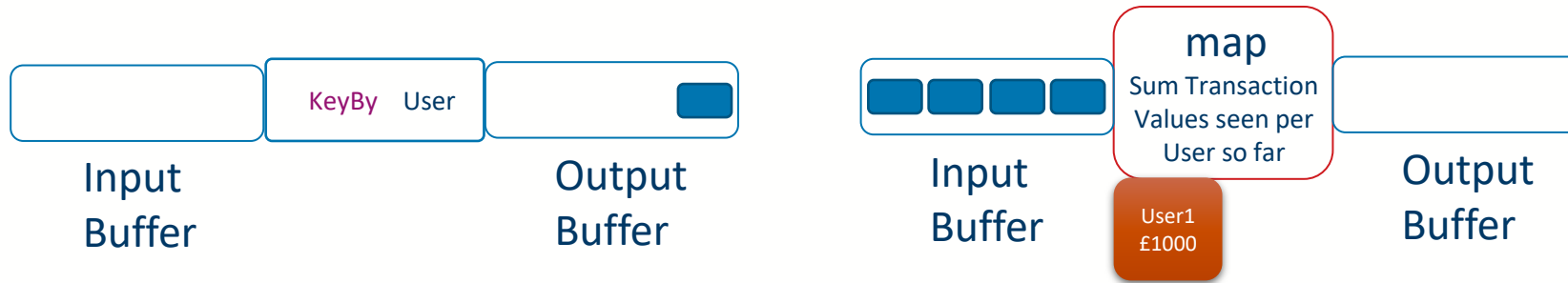


- New items will arrive over time and will be placed in the in-buffer
- If items arrive faster than the task can process them, the buffer will fill
- Once the buffer is full, it will stop allowing new items to be added



Buffering

- The other core problem we observe in streaming systems is buffering
 - Imagine each task has an in-buffer and an out-buffer

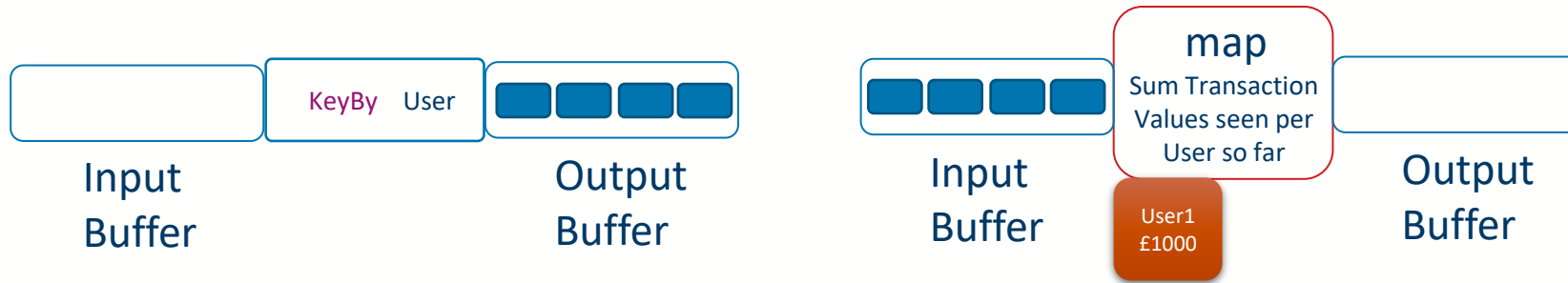


- This means that the previous task in the chain will not be able to empty its out-buffer



Buffering

- The other core problem we observe in streaming systems is buffering
 - Imagine each task has an in-buffer and an out-buffer

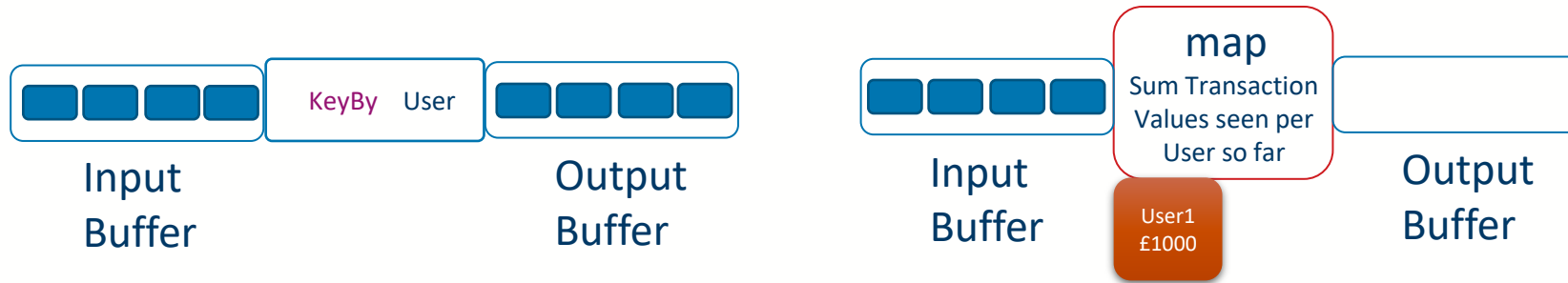


- This means that the previous task in the chain will not be able to empty its out-buffer
- Once its out buffer is full, it can process items...
- Meaning that its in-buffer will fill



Buffering

- The other core problem we observe in streaming systems is buffering
 - Imagine each task has an in-buffer and an out-buffer

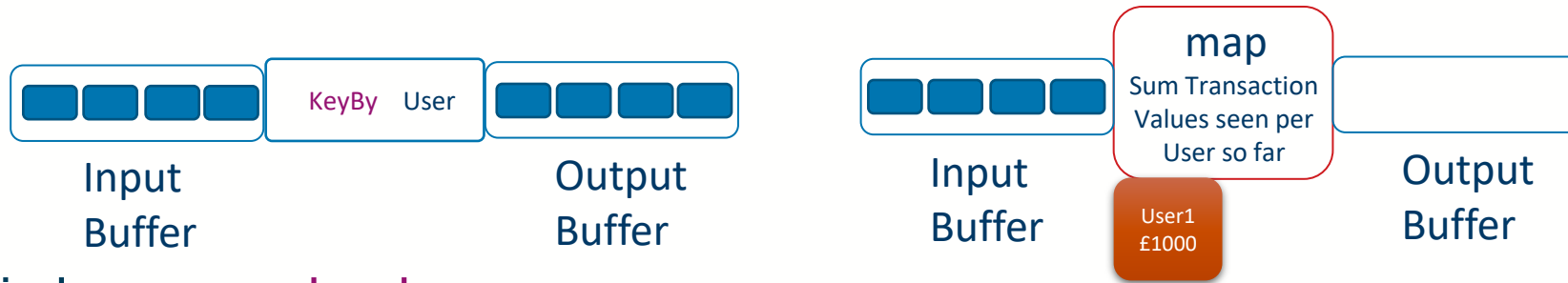


- This means that the previous task in the chain will not be able to empty its out-buffer
- Once its out buffer is full, it can process items...
- Meaning that its in-buffer will fill



Buffering

- The other core problem we observe in streaming systems is buffering
 - Imagine each task has an in-buffer and an out-buffer



- This is known as **back-pressure**
 - Think of it like a pipe that has filled with water
- Back pressure is bad, because
 1. It means we have items sitting in queues waiting to be processed
 2. Once all buffers in the system are full the job will usually fail, since a new item will arrive and there will be nowhere to put it



An Aside on Learning Streaming

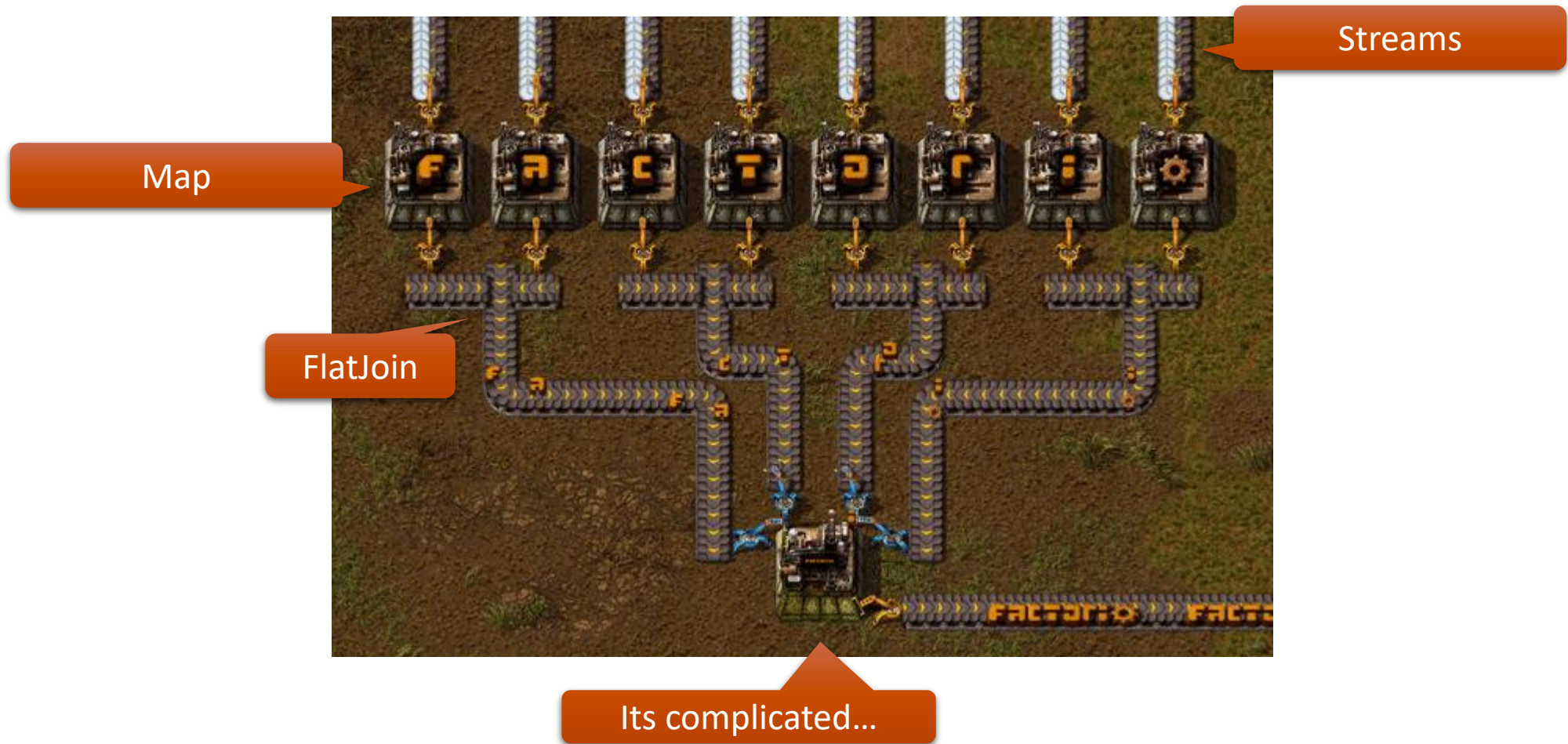
- We won't go deep into building streaming applications in this course due to time constraints, however if you are interested in learning how these systems work in a game environment then I would recommend the **Automation game genre**



- These games are about building incrementally more challenging factory production lines – which are **streaming systems**



An Aside on Learning Streaming





University
of Glasgow

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers



@UofGlasgow